Casa São Domingos Sávio: App de Gestão de Alunos

Programação para Dispositivos Móveis 2026.2 · IFPB Campus Esperança · Prof. André Atanasio
React Native + Expo SDK 58 · persistência local em SQLite (expo-sqlite)

1. Tema

A Casa São Domingos Sávio, de Campina Grande (PB), atende cerca de 60 crianças de 7 a 15 anos com oficinas, reforço escolar e refeições. As fichas dos alunos ficavam em papel, sem cópia de segurança. O app, baseado na minuta do projeto *Sincronize*, reúne tudo no celular e funciona sem internet.

Perfil	O que faz no app
Administrador	Tudo, inclusive cadastrar usuários.
Coordenador	CRUD da ficha do aluno, oficinas e matrículas, calendário anual (remarcar ou cancelar com justificativa), leitura dos relatórios e contato com os responsáveis, dashboard com exportação em PDF.
Professor	Vê só os dias das próprias oficinas, faz a lista de presença e escreve relatórios de comportamento.

Cada criança tem um histórico com frequência por oficina, relatórios de comportamento e uma linha do tempo que não pode ser alterada.

2. Modelo de dados

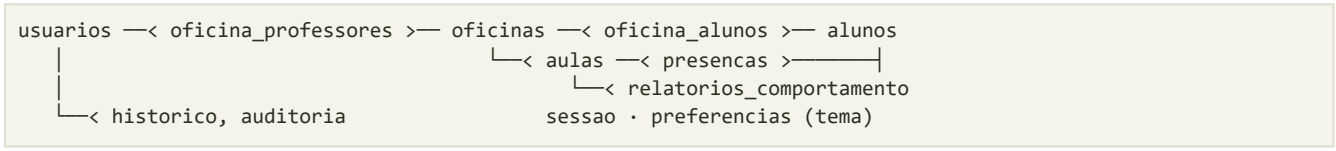


Tabela	Campos principais	Regras no banco
alunos (entidade principal)	Mais de 40 campos da ficha: dados pessoais, sociais, família, endereço, escola e saúde	NOT NULL em nome, nascimento, mãe, bairro e cidade; sim/não com CHECK (0,1); CPF único
usuarios	nome, email, senha_hash + senha_salt, perfil, telefone	Email único; senha nunca em texto puro (SHA-256 com salt)
oficinas e vínculos	nome, local; professores e alunos (N:N)	Chaves compostas impedem duplicidade
aulas	oficina, data, horário, status, data original, justificativa	Término depois do início; remarcar ou cancelar exige justificativa (CHECK)
presencas	aula, aluno, presente	Uma marcação por aluno por aula
relatorios_comportamento	aluno, aula, professor, texto, status	Trigger impede alterar o texto do professor
historico e **auditoria**	eventos do aluno; quem fez o quê e quando	Trigger impede alterar o histórico

3. Por que SQLite e não AsyncStorage

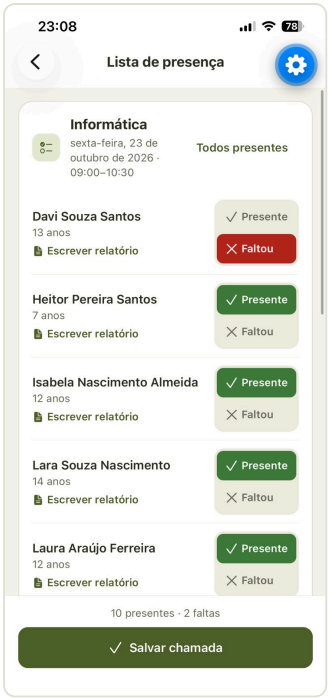
1. **Os dados são relacionais.** Alunos, oficinas, aulas, presenças e relatórios se ligam por chaves estrangeiras. No AsyncStorage (chave-valor) seria preciso guardar listas inteiras em JSON e manter as relações à mão.
2. **Consultas no banco.** Busca, filtros, ordenação e os totais do dashboard são feitos em SQL, com índices.
3. **Integridade garantida pelo banco.** NOT NULL, UNIQUE, CHECK, FOREIGN KEY e triggers valem mesmo se a interface falhar.
4. **Transações.** Cada cadastro grava ficha, histórico e auditoria juntos; se o app for interrompido, nada fica pela metade.
5. **Migrações versionadas** com PRAGMA user_version: o banco evoluiu da v1 para a v2 sem perder dados.

As telas (src/app) não escrevem SQL; chamam repositórios (src/database). As regras de negócio ficam em src/lib. São **37 testes automatizados** (npm test) que rodam o SQL de verdade, incluindo fechar e reabrir o banco.

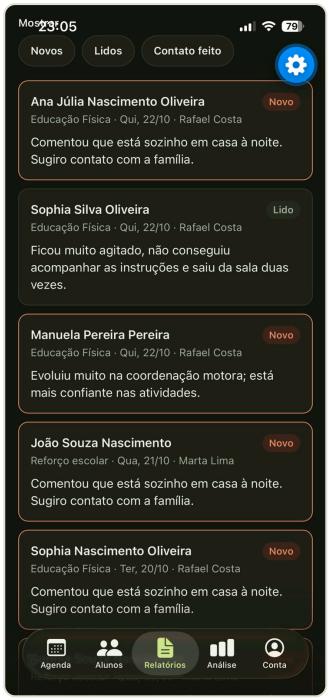
4. Capturas de tela



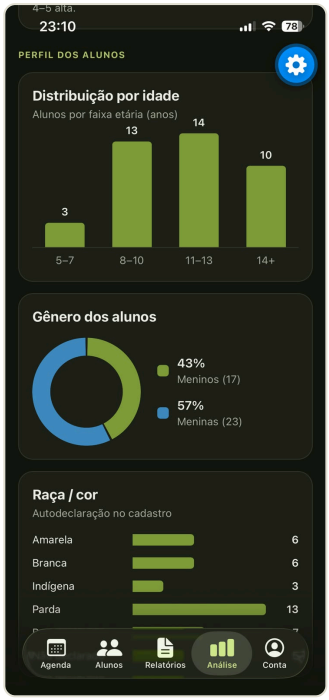
Agenda (coordenação)



Lista de presença (professor)



Relatórios de comportamento



Análise social

Evolução futura: sincronização entre dispositivos e backup na nuvem, previstos na minuta do *Sincronize*.